

Window Guard

Complete Help Manual - Version 0.5.0

Windows

Open Source

MIT License

Window Guard is a Windows privacy utility that protects selected desktop applications with a PIN. Protected applications remain registered after they close, and an application automatically returns to the **Locked** state when its running process ends. The next time it opens, Window Guard hides its visible window and asks for the PIN again.

Important: Window Guard is a privacy and convenience lock. It is not a replacement for Windows account security and is not designed to resist a Windows administrator, Task Manager, debugging tools, or a user who can modify the program configuration.

Contents

- 1. Overview
- 2. Requirements
- 3. Running and Installation
- 4. First Setup
- 5. Adding Protected Apps
- 6. Locked and Unlocked States
- 7. Persistent Relock Behavior
- 8. PIN Management
- 9. Manual Locking
- 10. System Tray
- 11. Start with Windows
- 12. Diagnostics
- 13. Menus and GitHub Integration
- 14. Configuration
- 15. Building the EXE
- 16. Firefox Portable
- 17. Troubleshooting
- 18. Forgotten PIN
- 19. Security Model
- 20. Antivirus / False Positives
- 21. Open Source and License

1. Overview

Window Guard monitors applications you explicitly add to its protected-app list. Each protected application has its own runtime lock state. Unlocking Firefox, for example, does not unlock Notepad++.

Feature	What it does
Persistent protection	An app remains registered until you choose Remove Selected .
Per-app PIN lock	Each protected app can be Locked or Unlocked independently.
Relock on close	When an unlocked app fully exits, it becomes Locked again for its next launch.
Window hiding	Window Guard hides visible top-level windows belonging to a locked protected app.
System tray	The main Window Guard interface can stay out of the way while protection continues.
Start with Windows	Optional per-user Windows startup entry.
Diagnostics	Displays process, PID, executable, and window information for troubleshooting.

2. Requirements

Running the compiled EXE

- Windows 10 or Windows 11
- No Python installation is required when using the standalone PyInstaller build.

Running from source

- Python 3.10 or newer
- `pystray`
- `Pillow`

```
python -m pip install -r requirements.txt
```

3. Running and Installation

Window Guard is intended to be portable. It does not require a traditional installer.

From source

```
python window_guard.py
```

You can also run `run_window_guard.bat`. The batch file checks for the required runtime packages.

Standalone EXE

Run `WindowGuard.exe`. Keep `WindowGuard_Help.html` and `WindowGuard_Help.pdf` in the same folder if you want the Help menu to open the manuals.

4. First Setup

1. Start Window Guard.
2. Open an application you want to protect, such as Firefox.
3. Click **Add Running App**.
4. Select the real application executable from the list.
5. Click **Set / Change PIN** and create a PIN.

6. The application is protected and is considered **Locked** by default.

Tip: Adding a running app is normally better than browsing for an EXE, because it identifies the executable that actually owns the visible window.

5. Adding Protected Applications

Add Running App

This is the recommended method. The selector displays:

- Window title
- Executable name
- PID
- Full executable path

Double-click a row or select it and choose **Add Selected App**.

Browse EXE

Use **Browse EXE...** when the target application is not currently running. Select the real executable file, not a launcher, updater, shortcut, or bootstrap program whenever possible.

Removing protection

Select an application in the protected-app list and click **Remove Selected**. This removes it from Window Guard's persistent protection list.

6. Locked and Unlocked States

State	Meaning
Locked	If the application is running or starts later, Window Guard hides its visible window and requires the PIN.
Unlocked	The correct PIN has been entered for the application's current running session. Its windows are allowed to remain visible.

The **Running** column is separate from the protection state. For example:

firefox.exe	Unlocked	Yes
notepad++.exe	Locked	No

7. Persistent Relock Behavior

This is the central behavior introduced in Window Guard v0.4.x.

```
Protected app starts
|
v
Window Guard detects it
|
v
Visible window is hidden
|
v
PIN is requested
|
v
Correct PIN -> app is Unlocked
|
v
User closes the app completely
|
v
Window Guard detects that the process ended
|
v
App automatically returns to Locked
|
v
Next launch requires the PIN again
```

Closing an application does **not** remove it from the protected list.

Restarting Window Guard also returns protected apps to the Locked runtime state. The list of protected apps itself remains stored in the configuration.

8. PIN Management

Use **Set / Change PIN** to create or replace the PIN.

- PIN length: 4 to 12 digits
- The PIN is not stored in plain text.
- Window Guard stores a random salt and a PBKDF2-HMAC-SHA256 derived hash.

- Changing an existing PIN requires the current PIN.

After repeated incorrect attempts, Window Guard temporarily disables PIN entry for a cooldown period.

9. Manual Locking

Lock Selected App

Select a protected application and click **Lock Selected App**. If it is currently running and visible, Window Guard hides its windows and asks for the PIN.

Lock All Now

Returns every protected app to Locked. Any visible protected windows are hidden.

The tray menu also includes **Lock All Now**.

10. System Tray

Closing the main Window Guard window minimizes it to the Windows notification area instead of stopping protection.

The tray menu provides:

- **Show Window Guard**
- **Lock All Now**
- **Exit**

Protection monitoring continues while the main window is hidden.

11. Start with Windows

Enable **Start Window Guard with Windows** to register Window Guard in the current user's startup registry key:

```
HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run
```

No administrator rights should normally be required for this setting. When launched by Windows startup, Window Guard starts hidden in the system tray.

Portable EXE note: If you move `WindowGuard.exe` after enabling Start with Windows, disable and re-enable the option so the startup entry points to the new location.

12. Diagnostics

Click **Diagnostics** when an application is not detected or is not being hidden correctly. The report includes:

- Protected application name and path
- Locked / Unlocked state
- Matched process IDs
- Visible top-level windows
- Executable names
- Window handles and titles

Use **Copy Report** to copy the diagnostic information for bug reports. Review the text before posting it publicly because window titles may contain names of websites, documents, or files you currently have open.

13. Menus and GitHub Integration

Window Guard v0.5.0 introduces a complete menu bar so the main actions are available both from buttons and from standard Windows menus.

File

- **Lock All Now** - immediately returns all protected applications to Locked.

- **Hide to Tray** - hides the main Window Guard window while monitoring continues.
- **Exit** - closes Window Guard and restores windows that it currently has hidden.

Applications

- **Add Running App...** - recommended method for adding a protected application.
- **Browse EXE...** - selects an application executable manually.
- **Lock Selected App** - locks the selected protected application immediately.
- **Remove Selected** - removes the selected application from persistent protection.

Tools

- **Set / Change PIN** - creates or replaces the Window Guard PIN.
- **Diagnostics** - shows process and window information used for troubleshooting.
- **Open Configuration Folder** - opens `%APPDATA%\WindowGuard`.
- **Open Application Folder** - opens the folder containing `WindowGuard.exe` or the Python source.
- **Start Window Guard with Windows** - enables or disables the per-user Windows startup entry.

GitHub

The GitHub menu links directly to the official open-source repository:

```
https://github.com/gexos/window-guard
```

- **Open Repository** - opens the project source code.
- **Latest Release** - opens the latest published GitHub release.
- **All Releases** - opens the complete releases page.
- **Report Bug / Request Feature** - opens GitHub's issue-template chooser.
- **View Issues** - opens existing project issues.

- **Copy Repository URL** - copies the repository address to the clipboard.

Open-source workflow: Bug reports and feature requests should use the repository issue forms so reports contain the Windows version, Window Guard version, reproduction steps, and relevant diagnostics.

Help

- **Help Contents (HTML)** - opens `WindowGuard_Help.html` in the default browser.
- **User Manual (PDF)** - opens `WindowGuard_Help.pdf` in the default PDF viewer.
- **About Window Guard** - shows the version, author, license, project repository, project purpose, and security notice.

The HTML and PDF help files must be kept in the same directory as `WindowGuard.exe` for the Help menu to find them.

14. Configuration

User settings are stored in:

```
%APPDATA%\WindowGuard\config.json
```

The file contains the protected-app list, executable paths, startup preference, and PIN verification data. It does not contain the actual PIN in readable form.

Do not include your personal `config.json` in public GitHub commits or release packages.

15. Building WindowGuard.exe

Install build requirements

```
python -m pip install --upgrade -r requirements-build.txt
```

Build manually

```
python -m PyInstaller --noconfirm --clean --onefile --windowed --name WindowGuard window_guard.py
```

Build with the included batch file

Double-click `build_window_guard.bat`. The script builds the EXE and copies the HTML/PDF help files into the `dist` folder.

The release directory will contain:

```
dist\  
  WindowGuard.exe  
  WindowGuard_Help.html  
  WindowGuard_Help.pdf
```

16. Firefox Portable and Launcher Applications

Portable application launchers may not own the actual visible window. For example, `FirefoxPortable.exe` launches the real `firefox.exe`, and the browser window belongs to `firefox.exe`.

For Firefox Portable:

1. Open Firefox Portable.
2. Choose **Add Running App**.
3. Select the visible Firefox window whose executable is `firefox.exe`.

If you protect only a launcher EXE that does not own the visible window, Window Guard may not be able to hide the application correctly.

17. Troubleshooting

The app is running but Window Guard does not detect it

- Use **Add Running App** instead of Browse EXE.
- Confirm the executable listed for the visible window.
- Run Diagnostics and check the matched PIDs.
- If the target app runs as administrator, try running Window Guard as administrator as well.

The window briefly appears before being hidden

Window Guard monitors normal Windows desktop windows at short intervals. It does not block process creation at the operating-system security layer, so a very brief visual appearance may be possible before the window is hidden.

Help files do not open

Confirm these files are in the same folder as `WindowGuard.exe`:

```
WindowGuard_Help.html  
WindowGuard_Help.pdf
```

A GitHub menu item does not open

Window Guard uses the default web browser for GitHub links. Confirm Windows has a default browser configured and that the repository address is reachable.

The tray icon is missing

If running from source, install:

```
python -m pip install pystray pillow
```

Also check the Windows notification-area overflow panel.

Start with Windows stopped working after moving the EXE

Disable and re-enable **Start Window Guard with Windows** after moving the executable.

18. Forgotten PIN

Window Guard does not contain a PIN-recovery feature because the original PIN is not stored.

If you lose the PIN, the configuration can be reset by closing Window Guard and removing its configuration file:

```
%APPDATA%\WindowGuard\config.json
```

This removes the saved PIN data and protected-app list, so Window Guard will behave like a new installation.

The fact that a local user can terminate Window Guard or remove its configuration is part of the documented security limitation. Window Guard is not intended to defend against someone who already controls the Windows account.

19. Security Model and Limitations

Window Guard is intended to prevent casual access to selected application windows when the utility is running. It does not create an operating-system security boundary.

A user with enough Windows access may bypass it by:

- Ending `WindowGuard.exe` in Task Manager

- Deleting or modifying its configuration
- Using another Windows account
- Using administrative or debugging tools
- Running an alternate copy or installation of the protected application

For genuine workstation protection, use a Windows account password/PIN and lock the workstation with `Windows` + `L`.

20. Antivirus / False Positives

Standalone Window Guard builds are created with PyInstaller. Some antivirus engines occasionally flag PyInstaller-packaged applications even when the source code is legitimate.

For public releases:

- Publish the complete source code.
- Allow users to build the EXE themselves.
- Publish a SHA-256 hash for the official release binary.
- Do not tell users to disable their antivirus software.

21. Open Source and License

Window Guard is open-source software released under the MIT License.

Author: Giorgos Xanthopoulos **aka gexos**

GitHub: <https://github.com/gexos/window-guard>

The source code may be inspected, modified, forked, and redistributed according to the terms of the MIT License.

Window Guard Help Manual - v0.5.0

Copyright (c) 2026 Giorgos Xanthopoulos (aka gexos). Released under the MIT License.